

2.4 集合 (Set)

集合

```
suits = {  
    "♠A",  
    "♥A",  
    "♣A",  
    "♦A"}  
}
```

- 集合是**无序可变**，使用一对大括号界定，元素间以逗号分隔
- **元素无序，且不重复**
- 集合中的元素必须是可hash对象，即不可变对象，**不能是可变对象**

2.4.1 集合(set)的创建与删除

- 可通过set([iterable])创建集合对象，定义时可以包含重复的对象，但只会保留一个。另外 `a={}` 产生空字典对象而不是空集合
- 可通过set([iterable])函数创建一个set对象，其元素为iterable对象中的元素，同样重复的元素仅保留一个

```
>>> set1={1,2,3,4}
>>> set1
{1, 2, 3, 4}
>>> set2={2,3,1,4}
>>> set2
{1, 2, 3, 4}|
```

```
>>> a={3,7,5,5}
>>> a
{3, 5, 7}
>>> a = {3,[1,2]}
TypeError: unhashable type:
'list'
```

```
>>> a=set(range(8,14))
>>> a
{8, 9, 10, 11, 12, 13}
>>> b
=set([0,1,2,3,0,1,2,3,7,8])
>>> b
{0, 1, 2, 3, 7, 8}
>>> c = set() #空集合
```

- 使用`del`删除整个集合

2.4.2 集合的运算:比较运算和内置函数

- 比较：
 - `==` `!=` 判断两个集合是否相等
 - `<` `>` 判断前一集合是否是后者的真子集和真超集
 - `<=` `>=` 判断前一集合是否是后者的子集和超集
- 元素是否存在：
 - `x in s` 或者 `x not in s`: 判断元素x是否在集合s中
- `len(s)`: 集合的元素个数
- `max(s)/min(s)/sum(s)`: 集合中元素最大值、最小值和求和

2.4.2 集合的运算:并, 交, 差……

方法	说明	例子
$s1 \mid s2 \mid \dots$ <code>s1.union(s2, ...)</code>	并集, $s1 \cup s2 \cup \dots$ 元素在任一个集合	$\{1,2,3\} \mid \{2,4\}$, return $\{1,2,3,4\}$ #新的集合 <code>{1,2,3}.union({2,4})</code>
$s1 \& s2 \& \dots$ <code>s1.intersection(s2, ...)</code>	交集, $s1 \cap s2 \cap \dots$ 元素在所有集合出现	$\{1,2,3\} \& \{2,4\}$, return $\{2\}$ #新的集合 <code>{1,2,3}.intersection({2,4})</code>
$s1 - s2$ <code>s1.difference(s2)</code>	差集, $s1 - s2$ 元素在s1但不在s2	$\{1,2,3\} - \{2,4\}$ return $\{1,3\}$ #新的集合 <code>{1,2,3}.difference({2,4})</code>
$s1 \wedge s2$ <code>s1.symmetric_difference(s2)</code>	对称差集, $s1 \oplus s2$ 元素在只属于其中一个 但不属于其中另一个 $= (s1 \cup s2) - s1 \cap s2$	$\{1,2,3\} \wedge \{2,4\}$ return $\{1, 3, 4\}$ #新的集合 <code>{1,2,3}.symmetric_difference({2,4})</code>
<code>s1.isdisjoint(s2)</code>	没有共同元素为True	<code>{1,2,3}.isdisjoint({2,4})</code> return False
<code>s1.issubset(s2)</code> $s1 \leq s2$	s1是否为s2的子集	<code>{1,2}.issubset({1,2,4})</code> return True
<code>s1.issuperset(s2)</code> $s1 \geq s2$	s1是否为s2的超集	<code>{1,2,4}.issuperset({1,2})</code> return True

实验:

- 集合的运算:并, 交, 差

```
>>> a = set('abracadabra')
>>> b = set('alacazam')
>>> a
>>> a - b
>>> a | b
>>> a & b
>>> a ^ b
```

```
# 集合a中包含而集合b中不包含的元素
# 集合a或b中包含的所有元素
# 集合a和b中都包含了的元素
# 不同时包含于a和b的元素
```

集合元素的更改(原地修改)

$s1 = \{1, 2, 3\}$
 $s2 = \{2, 3, 4\}$

方法	说明	例子
<code>s.add(x)</code>	将x添加到集合s	<code>s1.add('a')</code> # $s1 = \{1, 2, 3, 'a'\}$
<code>s1.update(s2)</code>	并集 类似 $s1 = s1 \cup s2$ (但改变自身 s1是同一对象, 下同)	<code>s1.update(s2)</code> # $s1 = \{1, 2, 3, 4\}$
<code>s1.intersection_update(s2)</code>	交集类似 $s1 = s1 \cap s2$	<code>s1.intersection_update(s2)</code> # $s1 = \{2, 3\}$
<code>s1.difference_update(s2)</code>	差集类似 $s1 = s1 - s2$	<code>s1.difference_update(s2)</code> # $s1 = \{1\}$
<code>s1.symmetric_difference_update(s2)</code>	对称差集类似 $s1 = s1 \oplus s2$	<code>s1.symmetric_difference_update(s2)</code> # $s1 = \{1, 4\}$
<code>s.remove(x)</code>	从集合s中移除x, 不存在抛异常KeyError	<code>s1.remove(1)</code> # $s1 = \{2, 3\}$
<code>s.discard(x)</code>	从集合s中移除x, 不存在不抛异常	<code>s1.discard(1)</code> # $s1 = \{2, 3\}$
<code>s.pop()</code>	从集合s移除一个元素并返回, 空时KeyError	<code>s1.pop()</code> # return 1, $s1 = \{2, 3\}$
<code>s.clear()</code>	清空集合s	<code>s1.clear()</code> # $s1 = \text{set}()$

实验

- 集合元素的更改

2.4.2 集合操作例子：去除list重复元素

```
import random
```

```
listRandom = [random.choice(range(10000)) for i in range(100)]
```

方法一：

```
noRepeat = []
for i in listRandom :
    if i not in noRepeat :
        noRepeat.append(i)
print(noRepeat)
```

方法二：

```
noRepeat = []
[noRepeat.append(i) for i in
listRandom
if not i in noRepeat]
```

注意以上代码列表推导式本身生成的列表没有任何意义【所有的元素都是None】

2.4.2 集合操作例子：去除list重复元素

方法三：

```
newSet = set(listRandom)  
noRepeat = list(newSet)
```

方法四：

```
newDict = dict.fromkeys(listRandom)  
noRepeat = list(newDict.keys())
```

- 以上方法利用set和dict的key的不重复特性去除重复元素，但是无序特性导致产生的list不是按照原来的顺序
- 利用sort或sorted方法的参数key来恢复原来的（下标）顺序
 - key为某个函数对象
 - 调用 key(value)返回的值来确定顺序

方法五：

```
noRepeat.sort(key=listRandom.index)  
newList = sorted(noRepeat, key = listRandom.index)
```

Set推导式（解析式）

- {2*x for x in range(5)}

```
>>> {2*x for x in range(5)}  
{0, 2, 4, 6, 8}
```

注意和字典推导式（解析式）的区别

```
>>> {k:v for k,v in zip(['a','b','c'],[1,2,3])}  
{'a': 1, 'b': 2, 'c': 3}
```

```
>>> {k:v for k,v in [('a',1),('b',2),('c',3)]}  
{'a': 1, 'b': 2, 'c': 3}
```

不可变集合frozenset

- set的元素可变
- frozenset为不可变集合对象，只能通过frozenset()函数来创建，与set()函数类似
- 除了集合元素的更改类的操作不支持外，set的其他操作都可应用于frozenset。

```
>>> help(frozenset)
Help on class frozenset in module builtins:
```

```
class frozenset(object)
|   frozenset() -> empty frozenset object
|   frozenset(iterable) -> frozenset object
```

```
>>> set1 = frozenset(range(10))
>>> set2 = frozenset("abc")
>>> set1
frozenset({0, 1, 2, 3, 4, 5, 6, 7, 8, 9})
>>> set2
frozenset({'c', 'a', 'b'})
>>> set1 | set2
frozenset({0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 'c', 'a', 'b'})
>>>
```

序列

	可变	不可变
有序	列表	元组**、字符串
无序	字典、集合	

- 字符串：连续字符，丰富的字符串处理方法
- 元组：有序的多个元素，不可变
- 列表：有序的多个元素，通过下标访问，可变
- 字典：无序，可以通过key访问
- 集合：无序，不重复，集合运算

2.5 再谈内置方法

sorted()

2.5 再谈内置方法sorted()

- 列表对象提供了sort()方法支持原地排序,
- 内置函数sorted()返回新的列表, 可对元组、列表、字典、集合等迭代对象、zip, enumerate等迭代器进行排序, 借助于其key参数可以实现自定义的的排序。
 - 调用 key(element)返回的值来确定顺序

2.5 再谈内置方法sorted()

```
>>> persons = [{'name': 'Dong', 'age': 37}, {'name': 'Zhang', 'age': 40}, {'name': 'Li', 'age': 50},  
{ 'name': 'Dong', 'age': 43}]  
>>> print(persons)  
[{'name': 'Dong', 'age': 37}, {'name': 'Li', 'age': 50}, {'name': 'Zhang', 'age': 40}, {'name': 'Dong', 'age':  
43}]
```

#使用key来指定排序依据，先按姓名升序排序，姓名相同的按年龄降序排序

```
def sort_by_name_age(x):  
    return x['name'], -x['age']  
print(sorted(persons, key=sort_by_name_age))
```



对于persons中的每个元素element
调用该函数sort_by_name_age(element)
该函数返回值为元组对象
(element['name'], -element['age'])

```
>>> print(sorted(persons, key=lambda x:(x['name'], -x['age'])))  
[{'name': 'Dong', 'age': 43}, {'name': 'Dong', 'age': 37}, {'name': 'Li', 'age': 50}, {'name': 'Zhang', 'age':  
40}]
```

2.5 再谈内置方法sorted()

- 以上例子都是自己定义函数来作为sorted的key
- **operator**模块的itemgetter函数：itemgetter(item) itemgetter(*items)
itemgetter返回的是一个**函数对象f**，该函数对象f在被调用【即f(obj)实参为obj】时，将调用对应对象obj的__getitem__(item)方法，即**obj[item]**

f = itemgetter(2)

f(obj)将返回 obj[2]

g=itemgetter(2,5,3)

g(obj)将返回元组

(obj[2],obj[5],obj[3])

attrgetter(attrName)或者
attrgetter(*attrNames)与
itemgetter类似，只是访问的是对象的属性，即obj.attrName（**不作要求**）

```
>>> phonebook = {'Linda':'7750', 'Bob':'9345',  
'Carol':'5834'}
```

```
>>> from operator import itemgetter
```

```
>>> sorted(phonebook.items(), key=itemgetter(1))
```

```
#按字典中元素的值进行排序，key(('Carol', '5834'))返回  
(('Carol', '5834'))[1]
```

```
[('Carol', '5834'), ('Linda', '7750'), ('Bob', '9345')]
```

```
>>> sorted(phonebook.items(), key=itemgetter(0))
```

2.5 再谈内置方法sorted()

```
>>> gameresult = [['Bob', 95.0, 'A'], ['Alan', 86.0, 'C'], ['Mandy', 83.5, 'A'], ['Rob', 89.3, 'E']]
```

```
>>> sorted(gameresult, key=itemgetter(0, 1)) #按姓名升序, 姓名相同按分数升序排序  
key([ 'Alan' , 86.0, 'C' ])返回元组(['Alan', 86.0, 'C' ][0], ['Alan', 86.0, 'C' ][1])  
[['Alan', 86.0, 'C'], ['Bob', 95.0, 'A'], ['Mandy', 83.5, 'A'], ['Rob', 89.3, 'E' ]]
```



```
>>> sorted(gameresult, key=itemgetter(1, 0)) #按分数升序, 分数相同的按姓名升序排序  
key([ 'Alan' , 86.0, 'C' ])返回元组(['Alan', 86.0, 'C' ][1], ['Alan', 86.0, 'C' ][0])  
[['Mandy', 83.5, 'A'], ['Alan', 86.0, 'C'], ['Rob', 89.3, 'E'], ['Bob', 95.0, 'A']]
```



```
>>> sorted(gameresult, key=itemgetter(2, 0)) #按等级升序, 等级相同的按姓名升序排序  
key([ 'Alan' , 86.0, 'C' ])返回元组(['Alan', 86.0, 'C' ][2], ['Alan', 86.0, 'C' ][0])  
[['Bob', 95.0, 'A'], ['Mandy', 83.5, 'A'], ['Alan', 86.0, 'C'], ['Rob', 89.3, 'E']]
```



2.5 再谈内置方法sorted()

```
>>> gameresult = [{'name':'Bob', 'wins':10, 'losses':3, 'rating':75.0},  
                  {'name':'David', 'wins':3, 'losses':5, 'rating':57.0},  
                  {'name':'Carol', 'wins':4, 'losses':5, 'rating':57.0},  
                  {'name':'Patty', 'wins':9, 'losses':3, 'rating':72.8}]
```

key({'name':'Bob', 'wins':10, 'losses':3, 'rating':75.0})返回

元组({'name':'Bob', 'wins':10, 'losses':3, 'rating':75.0}['wins'], {'name':'Bob', 'wins':10,
'losses':3, 'rating':75.0}['name'])

```
>>> sorted(gameresult, key=itemgetter('wins', 'name'))
```

#按' wins' 升序, 该值相同的按' name' 升序排序

```
[{'wins': 3, 'rating': 57.0, 'name': 'David', 'losses': 5}, {'wins': 4, 'rating': 57.0, 'name':  
'Carol', 'losses': 5}, {'wins': 9, 'rating': 72.8, 'name': 'Patty', 'losses': 3}, {'wins': 10, 'rating':  
75.0, 'name': 'Bob', 'losses': 3}]
```

2.5 再谈内置方法sorted()

- 以下代码演示如何根据另外一个列表list1的值来对当前列表list2元素进行排序
 - 先用zip组合对应位置的元素对后排序（首先按list1其次list2）

```
>>> list1 = ["what", "I'm", "sorting", "by"]
>>> list2 = ["something", "else", "to", "sort"]
>>> pairs = zip(list1, list2)
>>> pairs = sorted(pairs)
>>> pairs
[('I'm', 'else'), ('by', 'sort'), ('sorting', 'to'), ('what', 'something')]
>>> result = [x[1] for x in pairs]
>>> result
['else', 'sort', 'to', 'something']
```